

PART I · DATA LAYER

1 Single-Leader Replication
One writer, many read replicas. Total write order, no conflicts.

- + simple, strong ordering, mature tooling
- - writes capped at 1 node; lag → stale reads; failover gap + split-brain (use fencing)

USE default unless you must scale writes or survive region loss

2 Multi-Leader (Active-Active)
Every region accepts writes. Low local latency, write availability in partitions.

- **Conflicts:** LWW (lossy), CRDTs (merge), version vectors (app resolves)
- + survives region failure, low write latency
- - conflict resolution unavoidable; weaker consistency; complex

USE only for true multi-region write availability

3 Leaderless / Quorum
Write all, succeed on W of N; read R s.t. R+W>N. Dynamo model (Cassandra).

- **Repair:** read-repair + Merkle-tree anti-entropy
- + no failover cliff, tunable per-op consistency
- - eventual by default; conflicts → app; r/w amplification

4 Sharding
Split by partition key — hash / range / directory. Only way to scale writes.

- **Key is everything:** even spread + co-locate; bad key → hot partition
- - cross-shard = scatter-gather + lost ACID; resharding is major

USE when one node is the proven bottleneck

5 CQRS
Split write model from read model, synced via events. Reads tailored per screen.

- + independent scaling; many read shapes off one stream
- - eventual consistency; more infra; overkill on CRUD

USE for big read/write asymmetry - don't CQRS a contact form

6 Event Sourcing
Store the event log, not state. State = replay; snapshot to speed up.

- + full audit trail, time-travel, retroactive projections
- - event-versioning forever; unbounded log; steep curve

USE when auditability is a hard requirement

7 Outbox
Write event to outbox table in the same txn as the data; relay publishes. Kills dual-write.

- **Delivers:** at-least-once → consumers must be idempotent
- + guaranteed publish if data changed; any DB
- - relay infra + latency; table cleanup

8 Change Data Capture
Tap the DB log (binlog/WAL) → stream every row change (Debezium → Kafka). No app changes.

- + complete, ordered stream; catches legacy/manual writes
- - couples to physical schema; heavy initial snapshot

USE to sync search index / cache / read model

9 Materialized View
Pre-compute the slow query; read = lookup.

- **Refresh:** full (simple/stale) · incremental (cheap/hard) · event-driven (fresh/infra)
- - staleness + storage duplication

PART II · APP & WORKFLOW

10 Notification Delivery
One event → multi-step, multi-day sequence (email → push → remind), survive crash.

- **A Workflow engine:** durable instance (Temporal/Step Fn). +readable, durable -heavy, costly at volume
- **B Event + timer continuation:** each step registers next at timer service. +cheap, scales, multi-day -flow implicit, DIY dedup
- **C Fan-out per channel:** topic → email/push/SMS workers. +parallel, isolated -no sequencing

PICK audited → A · high-volume delay → B · real systems = B+C

11 Counter (Distributed)
Hot count — single row on millions of writers = lock contention.

- **A Sharded:** N sub-counters, sum on read. +exact, no hotspot -read aggregates N
- **B Approximate:** HLL / Count-Min. +tiny memory -not exact
- **C Buffered:** batch in Redis, flush. +huge throughput -loss on crash

PICK exact → A · analytics → B · burst → C · quota → Redis atomic

12 Feed / Timeline

Assemble feed at write-time or read-time?

- ▶ **A Fan-out write (push):** +fast reads –celebrity write-explosion
- ▶ **B Fan-out read (pull):** +cheap writes –slow reads
- ▶ **C Hybrid:** push normal + pull celebrities

PICK hybrid (C) is the production answer

13 Booking / Reservation

Exclusive resource, no double-book, don't hold inventory during slow checkout.

- ▶ **A Pessimistic lock:** +airtight –poor concurrency, bad for flash sales
- ▶ **B Optimistic hold + expiry:** HELD → CONFIRMED, timer reclaims. +high concurrency
- ▶ **C TCC / Saga:** try → confirm/cancel across services

PICK contention → B · multi-service → C · often B inside C

14 Job Scheduling & Triggering

Pipeline: Trigger → **Dispatch** → Queue → Execute. Dispatch is the exactly-once choke point.

- ▶ **Trigger:** cron (tz/DST!) · event · DAG dep · manual · self-schedule
- ▶ **Dispatch:** leader election · DB lease (SKIP LOCKED) · sharded
- ▶ **Execute:** queue + workers; at-least-once + idempotent; lease + heartbeat; reap zombies
- ▶ **Fairness:** priority queues · weighted-fair · rate-limited dispatch
- ▶ **Misfire:** fire-once · skip · backfill-all (△ stampede); overlap: allow/skip/queue
- ▶ **Workflow:** DAG, fan-out/fan-in, failure propagation

PICK few jobs → leader · many → DB lease · huge volume → sharded

15 Timer / Delayed Exec

Fire X at future time, millions pending, survive restart.

- ▶ **A Timer wheel:** O(1), in-memory. –not durable
- ▶ **B Durable store / delay queue:** poll `fire_time ≤ now`. +durable, any delay
- ▶ **C Hybrid:** persist + wheel for near-future precision

PICK in-proc → A · business delay → B · scale + precise → C

16 Rate Limiter

Cap requests / window — overload, fairness, quotas.

- ▶ **A Token bucket:** bursts + sustained rate (most used)
- ▶ **B Windows:** fixed (boundary burst) · sliding-log (precise/heavy) · sliding-counter (compromise)
- ▶ **C Distributed:** central Redis (exact/hop) · local-share (fast/loose) · gossip · cell

PICK burst → A · exact global → Redis · scale → local-share

17 Idempotency

Retries are inevitable. Key per op → store result → return on replay (Stripe-style).

- ▶ natural ops (SET/DELETE) need no key; relative ops (+, send) do
- ▶ **race:** unique constraint / atomic insert; TTL keys

NOTE prerequisite for at-least-once, outbox, sagas

18 Saga

Cross-service txn, no distributed ACID. Local steps + compensations; semantic undo (refund).

- ▶ **Choreography:** events chain. +simple –implicit flow
- ▶ **Orchestration:** central driver. +visible –coordinator
- ▶ – intermediate states visible; some acts can't be undone

19 Leaderboard / Top-K

Real-time ranking — use a sorted set (Redis ZSET), O(log N).

- ▶ scale → shard or per-window + merge
- ▶ trending → timestamped events + decay function

LESSON reach for the right primitive first

20 Search & Autocomplete

Free-text + prefix suggest in ms.

- ▶ **Search:** inverted index (ES/Solr), synced via CDC; BM25 ranking
- ▶ **Autocomplete:** trie + precomputed top-per-prefix cache
- ▶ – index is a 2nd copy → eventual consistency

21 Circuit Breaker

Stop calling a sick dependency.

- ▶ **States:** Closed (count) → Open (fail fast) → Half-open (trial)
- ▶ pair with fallback (cache/default/degrade); tune per dependency

22 Retry + Backoff + Jitter

Transients resolve on retry — but synced retries = storm.

- ▶ exponential backoff + **full jitter** (the default)
- ▶ only idempotent ops; cap attempts; **retry budget**
- ▶ pair with circuit breaker

23 Bulkhead Isolation

One flooded compartment ≠ whole ship. Per-dependency pools.

- ▶ thread/connection/semaphore pools; or cells/swimlanes
- ▶ + contains blast radius – less sharing efficiency

24 Load Shedding / Admission

Under overload, reject some so the rest succeed.

- ▶ shed low-priority first; signal on real saturation (queue/latency)
- ▶ respond **429 + Retry-After**; clients must honor

WHY partially up > fully down (brownout, not outage)

25 Cell-Based Architecture

Independent stack per user subset → failure hits one cell, not all.

- ▶ route by ID/tenant/geo; deploy cell-by-cell
- ▶ **Shuffle sharding:** random node combos shrink shared-fate
- ▶ – many copies; routing layer is critical infra